Supervised Learning COMP0048 : Coursework 2

Jordan Sydney-Darling 20127163, Daniel Pulido 25164517

January 2026



1 Part I

1.1 One vs Rest

The One vs Rest method to generalise a two-class classifier to a k-class classifier works by maintaining k two-class classifier models that are trained in parallel in an online fashion. Each sub model treats the positive class as class $i \in \{1, \ldots, k\}$ and the negative class as the set of all other classes $j \neq i$, hence each sub model is itself a two-class classifier that learns a linear separator between the 'One' target class and the 'Rest' (i.e. whether a point belongs to the target class or not).

Training proceeds sequentially by iterating through the data and updating the models one example at a time. For each new data point, all k one vs rest sub models are evaluated to produce a score equivalent to how $confident$ each sub model is that the new data point belongs to its target class. Geometrically this is equivalent to quantifying how close the point is to the decision boundary associated with the model, i.e if the point is close to the decision boundary its less confident about its prediction than if the point was far away from the decision boundary. Note, scores may be positive or negative, with a large positive value indicating strong confidence that the point lies in the target class and a large negative value indicating strong confidence that it belongs to the 'Rest'.

This process yields k scores, one for each sub-model. Prediction of the global k class classifier is finalised by aligning its prediction with the most 'positively confident' sub-model, the most confident being the model with the greatest positive score/confidence.

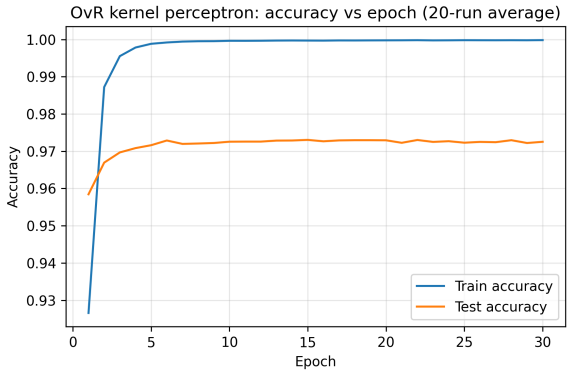
After scoring and the global prediction, each sub-model is updated immediately if its prediction (defined by the score) disagrees with the corresponding binary label for the current example. This online update process allows the classifiers to adapt incrementally as data are observed.

1.2 One vs Rest Kernel Perceptron Implementation

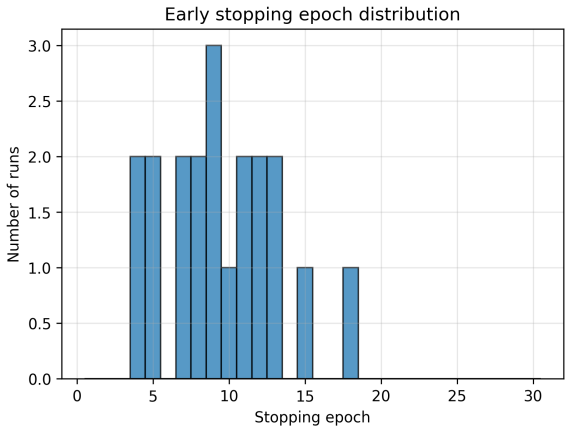
Algorithm 1 Online One-vs-Rest Kernel Perceptron

Input: Training data $\{(x_i, y_i)\}_{i=1}^{m}$, classes $\{1, \ldots, K\}$, kernel $K(\cdot, \cdot)$, epochs T
Initialise: For each class k, set $I^{(k)} = \emptyset$, $\alpha^{(k)} = \emptyset$
for $t = 1, \ldots, T$ **do**
  for $i = 1, \ldots, m$ **do**
    for $k = 1, \ldots, K$ **do**
      $s_k(x_i) = \sum_{j \in I^{(k)}} \alpha_j^{(k)} K(x_j, x_i)$
    end for
    $\hat{y} = \arg\max_k s_k(x_i)$ (score based)
    for $k = 1, \ldots, K$ **do**
      $$y_i^{(k)} = \begin{cases} +1 & \text{if } y_i = k \\ -1 & \text{otherwise} \end{cases}$$
      if $\text{sign}(s_k(x_i)) \neq y_i^{(k)}$ **then**
        $I^{(k)} \leftarrow I^{(k)} \cup \{i\}$
        $\alpha^{(k)} \leftarrow \alpha^{(k)} \cup y_i^{(k)}$
      end if
    end for
  end for
end for

To determine a suitable number of training epochs, we first investigated the convergence pattern of the OvR kernel perceptron algorithm. Initially the model was trained over 20 independent runs using 1-30 epochs, and the average train and test error was recorded. The resulting accuracy curves showed rapid convergence, with both training and test accuracy plateauing after a small number of epochs. The model was then again trained over 20 independent runs but an early stopping criteria was implemented. Upon completing each epoch pass the model was validated on a 10% validation subset that was held out from the training data. Training was terminated when the improvement in validation accuracy failed to improve by a tolerance of 10^{-3} for 3 epochs in a row.



(a)



(b)

The distribution of stopping epochs across runs concentrated primarily between approximately 4 and 13 epochs. Based on this analysis, number of epochs was fixed at 5 for all later experiments. This choice

captures the vast majority of meaningful learning across runs whilst also avoiding unnecessary computation beyond the point of meaningful improvements. It also ensures consistency and comparability across models and results in later experiments.

1.3 One vs Rest : Polynomial Kernel Perceptron

We first investigate the use of a polynomial kernel.

$$K(p,q) = (p \cdot q)^d \tag{1}$$

1.3.1 Basic Results

The perceptron was trained on a 80-20% train-test split using a polynomial kernel of degree $d = 1...7$. We performed 20 runs (each with a different train-test split) and evaluated the trained model on both the training set and the test set. The resulting errors were then averaged across all 20 runs and the mean and std recorded below.

d	mean train error rates (%)	mean test error rates (%)
1	7.509 ± 1.883	9.581 ± 1.831
2	0.722 ± 0.34	3.782 ± 0.691
3	0.196 ± 0.086	3.073 ± 0.52
4	0.074 ± 0.033	2.839 ± 0.456
5	0.054 ± 0.046	2.841 ± 0.371
6	0.032 ± 0.014	2.734 ± 0.541
7	0.033 ± 0.012	2.844 ± 0.395

Table 1

The results show that as the degree of the polynomial increases, the training error decreases, this is to be expected as the more complex the model the better it becomes at fitting the training data. The test errors however tell a different story. Initially at lower polynomial degree, test errors behave similar to the train errors, low degree fits poorly and increasing the degree decreases the error. This only applies up until a point, $d = 6$, where increasing the degree beyond this point actually causes the test error to increase again, this behaviour is characteristic of overfitting as the model starts to fit the noise rather than the actual patterns in the data.
We can conclude that the optimal degree is $d = 6$

1.3.2 Cross-Validation

To select the optimal polynomial degree we performed 20 runs of model fitting, this time each run included 5-fold cross validation for each polynomial degree to select the optimal parameter (degree) followed by retraining the model using the optimal parameter (d^*). The model was then again evaluated on the training set and the test set the results of which are listed below.

run	d*	test error rate (%)
1	7.0000	3.0923
2	5.0000	3.3745
3	5.0000	3.1192
4	5.0000	3.1057
5	5.0000	3.0116
6	7.0000	2.8502
7	4.0000	3.1191
8	5.0000	3.0519
9	6.0000	2.9848
10	6.0000	3.0385
11	5.0000	3.0116
12	5.0000	2.8772
13	5.0000	3.1863
14	7.0000	3.0251
15	4.0000	3.1460
16	4.0000	2.9444
17	5.0000	2.9442
18	4.0000	3.0653
19	4.0000	3.2401
20	5.0000	2.9174

Table 2

The results show that the optimal parameter has mean of 5.15±0.96307, train errors have a mean of 0.0511±0.037% and test errors have a mean of 2.712±0.316%. This aligns with the previous results without cross-validation where we identified the optimal parameter as being $d = 6$, as 6 is within the mean+std of the cross validation result.

1.3.3 Confusion Matrix

A confusion matrix was also computed at the same time as cross-validation. The results were averaged across 20 runs and they show us the rate which the model confused 2 labels. I.e if the true label was "7" and "2" was predicted then the confusion error for "(7,2)" would increase.

	0	1	2	3	4	5	6	7	8	9
0	0.000 ± 0.000	0.033 ± 0.099	0.095 ± 0.177	0.227 ± 0.181	0.193 ± 0.234	0.225 ± 0.267	0.225 ± 0.249	0.047 ± 0.150	0.113 ± 0.187	0.081 ± 0.140
1	0.021 ± 0.092	0.000 ± 0.000	0.057 ± 0.136	0.017 ± 0.076	0.237 ± 0.363	0.020 ± 0.087	0.270 ± 0.300	0.040 ± 0.121	0.076 ± 0.153	0.019 ± 0.082
2	0.430 ± 0.424	0.263 ± 0.377	0.000 ± 0.000	0.560 ± 0.574	0.571 ± 0.347	0.049 ± 0.215	0.277 ± 0.328	0.735 ± 0.486	0.394 ± 0.478	0.000 ± 0.000
3	0.227 ± 0.436	0.091 ± 0.218	0.767 ± 0.854	0.000 ± 0.000	0.095 ± 0.227	1.702 ± 1.406	0.027 ± 0.120	0.454 ± 0.596	0.907 ± 0.794	0.162 ± 0.344
4	0.000 ± 0.000	0.406 ± 0.481	0.577 ± 0.516	0.029 ± 0.128	0.000 ± 0.000	0.170 ± 0.317	0.438 ± 0.601	0.348 ± 0.433	0.000 ± 0.000	0.588 ± 0.557
5	0.831 ± 1.066	0.104 ± 0.248	0.444 ± 0.542	1.152 ± 0.891	0.278 ± 0.400	0.000 ± 0.000	0.729 ± 0.505	0.136 ± 0.272	0.315 ± 0.471	0.484 ± 0.496
6	0.760 ± 0.649	0.210 ± 0.347	0.125 ± 0.252	0.000 ± 0.000	0.439 ± 0.513	0.302 ± 0.401	0.000 ± 0.000	0.031 ± 0.135	0.174 ± 0.324	0.031 ± 0.136
7	0.056 ± 0.169	0.269 ± 0.401	0.348 ± 0.424	0.157 ± 0.274	0.819 ± 0.569	0.161 ± 0.351	0.000 ± 0.000	0.000 ± 0.000	0.411 ± 0.459	0.876 ± 0.774
8	0.680 ± 0.631	0.484 ± 0.688	0.434 ± 0.410	1.004 ± 0.839	0.604 ± 0.745	1.048 ± 0.835	0.319 ± 0.415	0.530 ± 0.693	0.000 ± 0.000	0.184 ± 0.397
9	0.183 ± 0.385	0.030 ± 0.130	0.151 ± 0.323	0.306 ± 0.452	1.193 ± 0.797	0.062 ± 0.187	0.060 ± 0.179	0.841 ± 0.935	0.095 ± 0.225	0.000 ± 0.000

Table 3

From the confusion matrix we can see that the model struggles at telling apart digits that look very similar, such as 8 and 3, 7 and 9, 5 and 6. We can also see that it does very well at telling apart digits that dont look similar at all.

1.3.4 Hard to Predict Samples

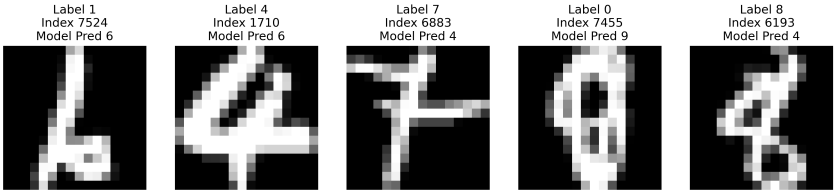


Figure 2

These samples are hard to predict as they are ambiguous, take the first image for example, it has an additional stroke that makes it look like a 6 even to the human eye when in reality its labeled as a 1. The 4th image is labeled as a 0 but is distorted and the final image is labeled as an 8 but the way its been drawn within the image it looks like a 4 is present. These are all examples where the images are ambiguous and so geometrically would sit very close to the decision boundary, hence the model is more likely to incorrectly predict their labels and hence are hard to predict.

1.4 One vs Rest: Gaussian Kernel Perceptron

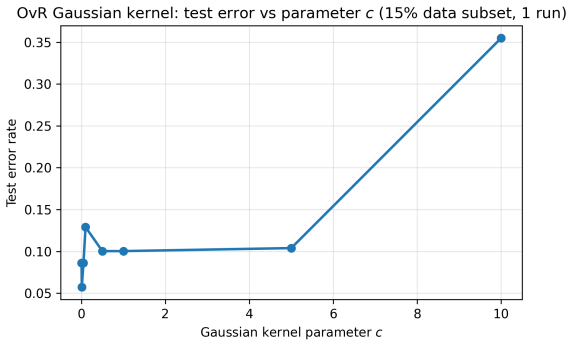
We now investigate the use of a gaussian kernel instead of a polynomial kernel.

$$K(p,q) = \exp\left(-c\,\|p-q\|^2\right) \tag{2}$$

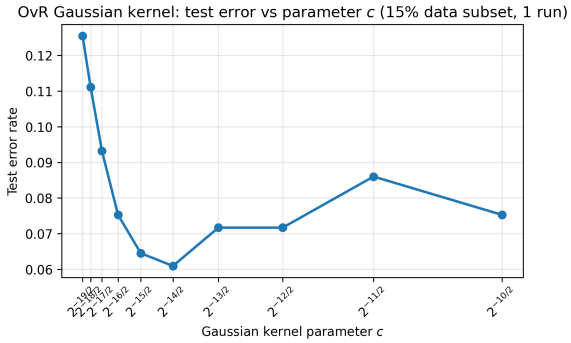
1.4.1 Parameter selection

To determine a suitable set S of candidate values for the Gaussian kernel parameter c, we conducted a series of exploratory experiments on a 15% subset of the full dataset. We first trained the model over a wide range of c values in order to identify the approximate scale at which the test error was minimised. This was followed by progressively restricting the range and adopting the form $c = 2^{-b/2}$. This allowed us to focus on the region of best performance while keeping computational cost low. Finally, the most promising range of c values was evaluated on the full dataset to confirm that the observed behaviour applied to the full dataset and not just the subset. Based on these experiments the final set S was chosen as

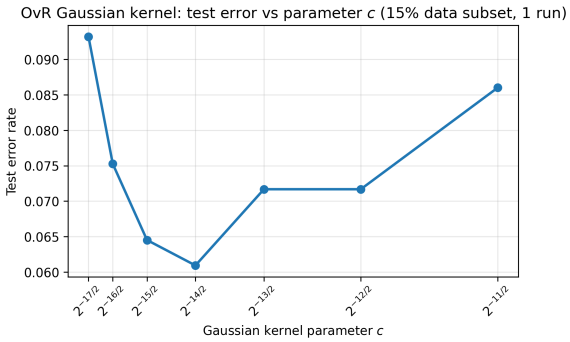
$$S = \left\{ 2^{-b/2} \;\middle|\; b \in \{9, 10, \ldots, 15\} \right\},$$



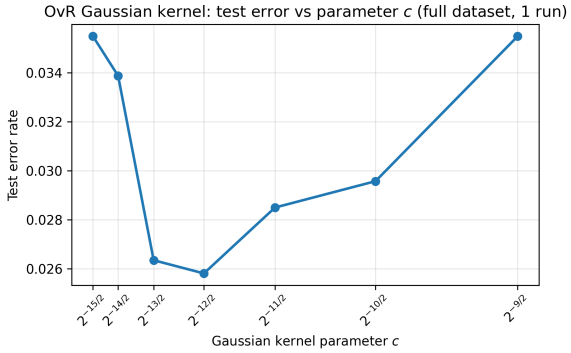
(a) c = [0.005, 0.01, 0.05, 0.1, 0.5, 1, 5, 10] on 15% of the dataset



(b) c = $[2^{-b/2}, b \in [10, 20)]$ on 15% of the dataset



(c) c = $[2^{-b/2}, b \in [11, 18)]$ on 15% of the dataset



(d) c = $[2^{-b/2}, b \in [9, 16)]$ on full dataset

Figure 3: Gaussian OVR parameter sweeps at different resolutions and dataset sizes

1.4.2 Basic Results

c	mean train error rates (%)	mean test error rates (%)
$2^{-9/2}$	0.024 ± 0.036	3.556 ± 0.526
$2^{-10/2}$	0.032 ± 0.042	2.968 ± 0.442
$2^{-11/2}$	0.025 ± 0.019	2.651 ± 0.371
$2^{-12/2}$	0.032 ± 0.015	2.543 ± 0.405
$2^{-13/2}$	0.05 ± 0.024	2.605 ± 0.296
$2^{-14/2}$	0.101 ± 0.076	2.919 ± 0.363
$2^{-15/2}$	0.35 ± 0.129	3.301 ± 0.527

Table 4

From the results above we observe the expected U shape in test errors, as the value of c decreases the test errors initially decrease up until the point $c = 2^{-12/2}$, after which decreasing c further causes test errors to increase. An interesting observation to note is that unlike in the polynomial case where training errors continued to decrease as c decreased, in the gaussian case the training errors also exhibited a U shape like the test errors. This is because in the with the gaussian kernel the width parameter defines locally the influence of training points are. If the width is too small, the model doesn't effectively capture the similarity between points, if its too wide data points too far away influence the model too strongly. Hence the extremes cause poor performance in both the test and training errors.

We conclude that the optimal parameter is $c = 2^{-12/2}$

1.4.3 Cross Validation

run	c*	test error rate (%)
1	0.0156	2.8906
2	0.0156	3.1729
3	0.0156	3.0788
4	0.0156	2.8637
5	0.0221	2.9175
6	0.0156	2.6755
7	0.0110	2.7560
8	0.0156	2.8367
9	0.0110	2.7562
10	0.0110	2.8368
11	0.0221	2.5679
12	0.0156	2.7831
13	0.0221	3.0251
14	0.0156	2.6888
15	0.0110	2.9174
16	0.0156	2.8906
17	0.0156	2.8770
18	0.0110	2.7561
19	0.0156	2.9040
20	0.0221	2.8233

Table 5

The results show that the optimal parameter has mean of $0.01575 \pm 0.0037037 = 2^{-11.97700872/2} \pm 0.0037037$, train errors have a mean of $0.0336 \pm 0.0183\%$ and test errors have a mean of $2.5242 \pm 0.3927\%$. This aligns with the previous results without cross-validation where we identified the optimal parameter as being $c = 2^{-12/2}$

1.4.4 Confusion error matrix

	0	1	2	3	4	5	6	7	8	9
0	0.000 ± 0.000	0.032 ± 0.096	0.145 ± 0.212	0.206 ± 0.181	0.142 ± 0.186	0.272 ± 0.349	0.129 ± 0.187	0.033 ± 0.099	0.064 ± 0.129	0.081 ± 0.140
1	0.021 ± 0.092	0.000 ± 0.000	0.077 ± 0.155	0.019 ± 0.081	0.234 ± 0.287	0.000 ± 0.000	0.234 ± 0.227	0.060 ± 0.143	0.096 ± 0.167	0.000 ± 0.000
2	0.311 ± 0.427	0.214 ± 0.263	0.000 ± 0.000	0.410 ± 0.502	0.551 ± 0.436	0.143 ± 0.313	0.142 ± 0.247	0.380 ± 0.417	0.388 ± 0.475	0.000 ± 0.000
3	0.274 ± 0.484	0.096 ± 0.231	0.650 ± 0.567	0.000 ± 0.000	0.000 ± 0.000	1.946 ± 1.399	0.000 ± 0.000	0.345 ± 0.314	0.973 ± 0.546	0.089 ± 0.213
4	0.087 ± 0.206	0.348 ± 0.493	0.512 ± 0.752	0.030 ± 0.130	0.000 ± 0.000	0.234 ± 0.287	0.261 ± 0.350	0.312 ± 0.455	0.117 ± 0.295	0.551 ± 0.587
5	0.797 ± 0.855	0.069 ± 0.206	0.451 ± 0.457	1.005 ± 0.815	0.378 ± 0.617	0.000 ± 0.000	0.553 ± 0.504	0.102 ± 0.243	0.489 ± 0.629	0.203 ± 0.431
6	0.775 ± 0.900	0.185 ± 0.284	0.212 ± 0.340	0.031 ± 0.135	0.498 ± 0.496	0.356 ± 0.431	0.000 ± 0.000	0.027 ± 0.118	0.237 ± 0.403	0.000 ± 0.000
7	0.000 ± 0.000	0.240 ± 0.348	0.313 ± 0.545	0.190 ± 0.356	0.997 ± 0.767	0.065 ± 0.196	0.000 ± 0.000	0.000 ± 0.000	0.591 ± 0.461	1.029 ± 0.874
8	0.677 ± 0.483	0.315 ± 0.527	0.432 ± 0.479	0.904 ± 0.918	0.432 ± 0.468	1.112 ± 0.803	0.178 ± 0.308	0.275 ± 0.503	0.000 ± 0.000	0.143 ± 0.287
9	0.029 ± 0.125	0.063 ± 0.188	0.182 ± 0.333	0.244 ± 0.442	0.875 ± 0.760	0.089 ± 0.211	0.000 ± 0.000	0.832 ± 0.550	0.150 ± 0.372	0.000 ± 0.000

Table 6

1.4.5 Hard to Predict Samples

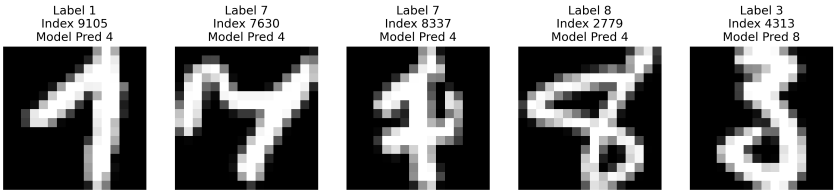


Figure 4

1.4.6 Comparison with Polynomial Kernel

Overall the Gaussian kernel achieves slightly better test performance than the polynomial kernel. Using cross validation the gaussian kernel has a mean test error of 2.52%, compared to 2.71% for the polynomial kernel.

The Gaussian kernel also has very low training error, this indicates a that the model fits the training data very well. However, its performance is sensitive to the choice of parameter c with very large or small values leading to poorer generalisation.

Also note that while the gaussian kernel shows slight improvement in test accuracy, this comes at the cost of increased computation.

1.5 One vs One

The One vs One method provides an alternative strategy for extending a two class classifier to a k class classifier. Instead of training one sub-model per class against all other classes, the OvO method trains a submodel for every pair of classes, ie each sub model learns the classify between class a and class b only, it does not learn on any other classes. For k classes this results in $\frac{k(k-1)}{2}$ binary classifiers, each specialised in distinguishing between a specific pair of classes.

At prediction time, each binary classifier votes for one of its two classes. The final predicted label is obtained by majority voting across all pairwise classifiers. Compared to the One vs Rest method, OvO has an increased computational cost due to the larger number of classifiers.

1.5.1 One vs One Kernel Perceptron Implementation

Algorithm 2 One-vs-One Online Kernel Perceptron

Input: $\{(x_i, y_i)\}_{i=1}^m$, classes $C = \{1, \ldots, K\}$, kernel $K(\cdot,\cdot)$, epochs T
Initialise: For all (a,b) with $a > b$, $\alpha^{(a,b)} = \emptyset$, $I^{(a,b)} = \emptyset$
for $t = 1, \ldots, T$ **do**
  for $i = 1, \ldots, m$ **do**
    Initialise votes $v_k = 0$
    for each pair (a,b) **do**
      $s_{a,b}(x_i) = \sum_{j \in I^{(a,b)}} \alpha_j^{(a,b)} K(x_j, x_i)$
      $v_a \leftarrow v_a + \mathbb{I}[s_{a,b}(x_i) \geq 0]$
      $v_b \leftarrow v_b + \mathbb{I}[s_{a,b}(x_i) < 0]$
    end for
    Predict $\hat{y} = \arg\max_k v_k$ (vote based)
    for each pair (a,b) with $y_i \in \{a,b\}$ **do**
      $$y_i^{(a,b)} = \begin{cases} +1 & \text{if } y_i = a \\ -1 & \text{otherwise} \end{cases}$$
      if $\text{sign}(s_{a,b}(x_i)) \neq y_i^{(a,b)}$ **then**
        $I^{(a,b)} \leftarrow I^{(a,b)} \cup \{i\}$
        $\alpha^{(a,b)} \leftarrow \alpha^{(a,b)} \cup y_i^{(a,b)}$
      end if
    end for
  end for
end for

1.6 One vs One : Polynomial Kernel Perceptron

1.6.1 Basic Results

d	mean train error rates (%)	mean test error rates (%)
1	4.562 ± 0.701	7.282 ± 1.031
2	0.735 ± 0.321	4.108 ± 0.495
3	0.218 ± 0.09	3.409 ± 0.476
4	0.13 ± 0.091	3.347 ± 0.469
5	0.097 ± 0.147	3.323 ± 0.534
6	0.045 ± 0.022	3.344 ± 0.522
7	0.061 ± 0.106	3.46 ± 0.426

Table 7

From the results above we observe the training errors decrease as the degree of the polynomial increases. We also observe expected U shape in test errors, as the value of d decreases the test errors initially decrease up until the point $d = 5$, after which decreasing d further causes test errors to increase as the model is overfitting to noise.

We conclude that the optimal parameter is $d = 5$

1.6.2 Cross validation

run	d*	test error rate (%)
1	5.0000	3.7107
2	4.0000	3.8316
3	5.0000	3.8317
4	4.0000	3.4150
5	4.0000	3.3209
6	5.0000	3.4417
7	4.0000	3.8720
8	3.0000	3.5897
9	4.0000	3.6302
10	4.0000	3.4686
11	4.0000	3.4148
12	6.0000	3.4687
13	5.0000	3.5762
14	6.0000	3.4687
15	4.0000	3.5762
16	4.0000	3.5360
17	3.0000	3.4417
18	5.0000	3.5224
19	4.0000	3.4955
20	6.0000	3.5224

Table 8

The results show that the optimal parameter has mean of 4.45±0.86458, train errors have a mean of 0.115±0.099% and test errors have a mean of 3.2796±0.5049%. This aligns with the previous results without cross-validation where we identified the optimal parameter as being $d = 5$

1.6.3 Confusion Matrix

	0	1	2	3	4	5	6	7	8	9
0	0.000 ± 0.000	0.000 ± 0.000	0.272 ± 0.233	0.225 ± 0.206	0.127 ± 0.183	0.243 ± 0.202	0.210 ± 0.278	0.000 ± 0.000	0.146 ± 0.245	0.031 ± 0.094
1	0.000 ± 0.000	0.000 ± 0.000	0.019 ± 0.084	0.019 ± 0.081	0.356 ± 0.560	0.038 ± 0.114	0.307 ± 0.308	0.020 ± 0.087	0.135 ± 0.218	0.000 ± 0.000
2	0.560 ± 0.456	0.218 ± 0.268	0.000 ± 0.000	0.551 ± 0.661	0.688 ± 0.668	0.147 ± 0.315	0.283 ± 0.342	0.571 ± 0.541	0.555 ± 0.626	0.082 ± 0.196
3	0.375 ± 0.497	0.062 ± 0.185	0.991 ± 0.974	0.000 ± 0.000	0.059 ± 0.177	2.159 ± 1.300	0.000 ± 0.000	0.222 ± 0.424	1.341 ± 0.923	0.090 ± 0.214
4	0.204 ± 0.279	0.526 ± 0.639	0.816 ± 0.511	0.058 ± 0.175	0.000 ± 0.000	0.140 ± 0.383	0.434 ± 0.544	0.255 ± 0.380	0.120 ± 0.240	0.508 ± 0.717
5	1.841 ± 1.522	0.000 ± 0.000	0.442 ± 0.530	1.669 ± 0.978	0.454 ± 0.540	0.000 ± 0.000	0.834 ± 0.641	0.103 ± 0.245	0.587 ± 0.666	0.178 ± 0.381
6	0.947 ± 0.672	0.273 ± 0.358	0.240 ± 0.480	0.000 ± 0.000	0.938 ± 0.585	0.486 ± 0.417	0.000 ± 0.000	0.000 ± 0.000	0.123 ± 0.248	0.000 ± 0.000
7	0.032 ± 0.141	0.457 ± 0.457	0.473 ± 0.566	0.146 ± 0.254	1.113 ± 0.784	0.086 ± 0.205	0.000 ± 0.000	0.000 ± 0.000	0.252 ± 0.309	0.925 ± 0.909
8	0.860 ± 0.774	0.782 ± 0.676	0.608 ± 0.656	1.213 ± 0.976	0.762 ± 0.841	1.280 ± 0.780	0.142 ± 0.362	0.551 ± 0.695	0.000 ± 0.000	0.144 ± 0.290
9	0.151 ± 0.262	0.124 ± 0.248	0.123 ± 0.246	0.377 ± 0.414	1.699 ± 1.403	0.154 ± 0.268	0.000 ± 0.000	0.942 ± 0.599	0.059 ± 0.177	0.000 ± 0.000

Table 9

1.6.4 Hard to Predict Samples

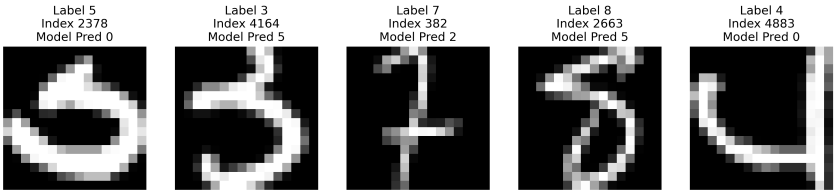


Figure 5

1.6.5 Comparison with One-vs-Rest

Comparing the One vs One and One vs Rest method using the polynomial kernel, OvO achieves slightly worse test performance overall. OvR has a mean test error of 2.71%, compared to 3.28% for OvO after cross validation. Additionally OvO is far more computationally expensive compared to OvR and as such for this dataset OvR would be the preferably method.